

Collective Multicast in a Fat Tree

Roger Marks
(EthAirNet Associates)

roger@EthAir.Net

2025-02-09

CONFIDENTIAL

k -ary Fat Tree

k is even

Each switch has k ports

k pods, each containing $k/2$ RS and $k/2$ FS

total of $k^2/2$ FS and $k^2/2$ RS

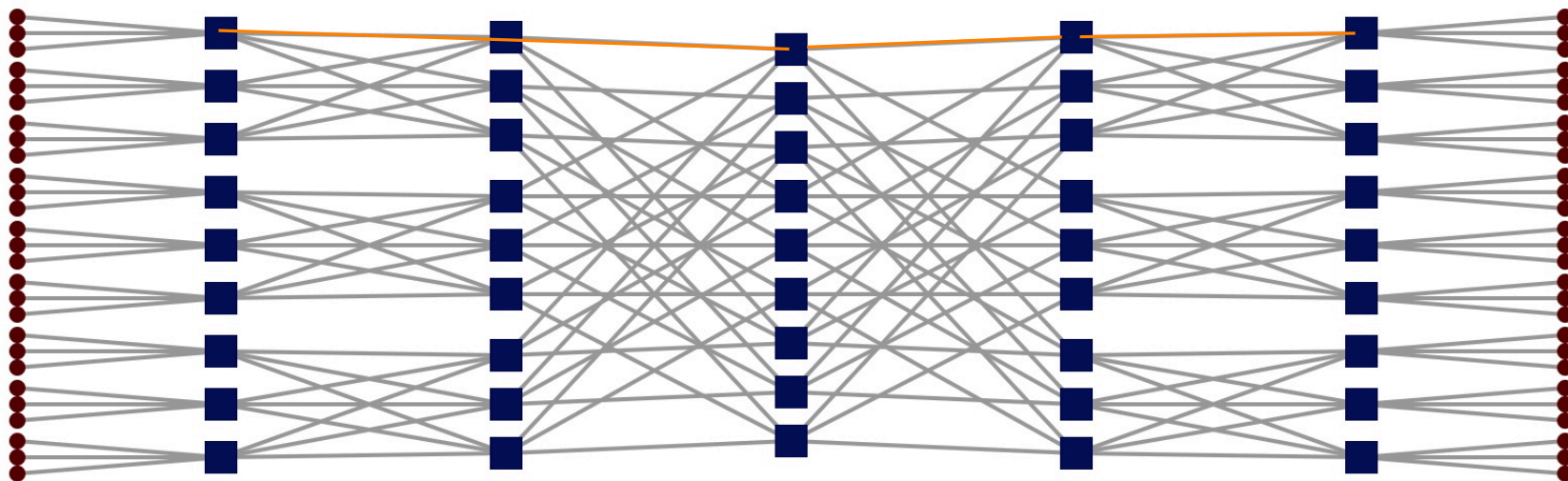
$k/2$ spines, each containing $k/2$ SS

total of $k^2/4$ SS

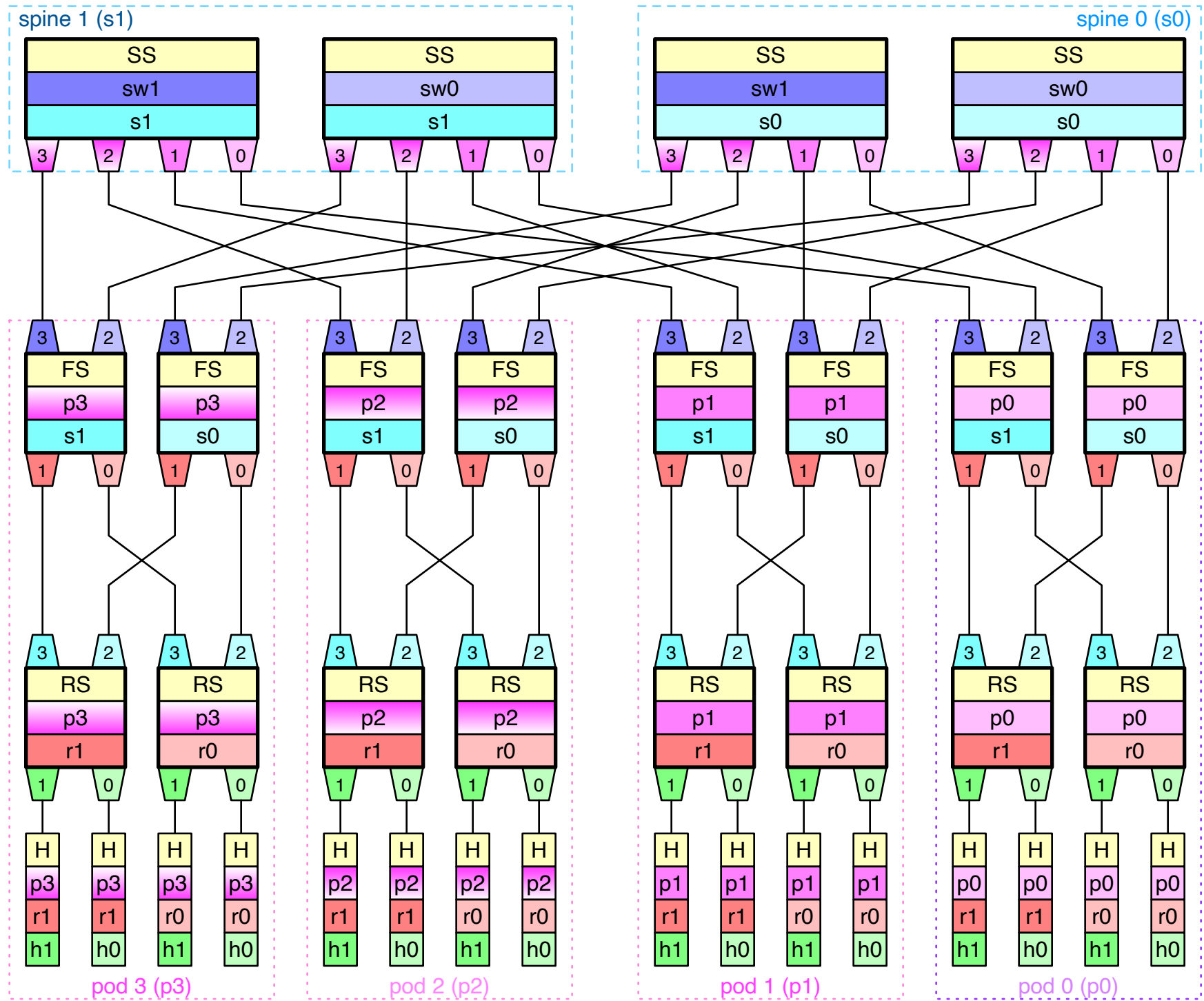
each RS has $k/2$ hosts

total of $k^3/4$ hosts

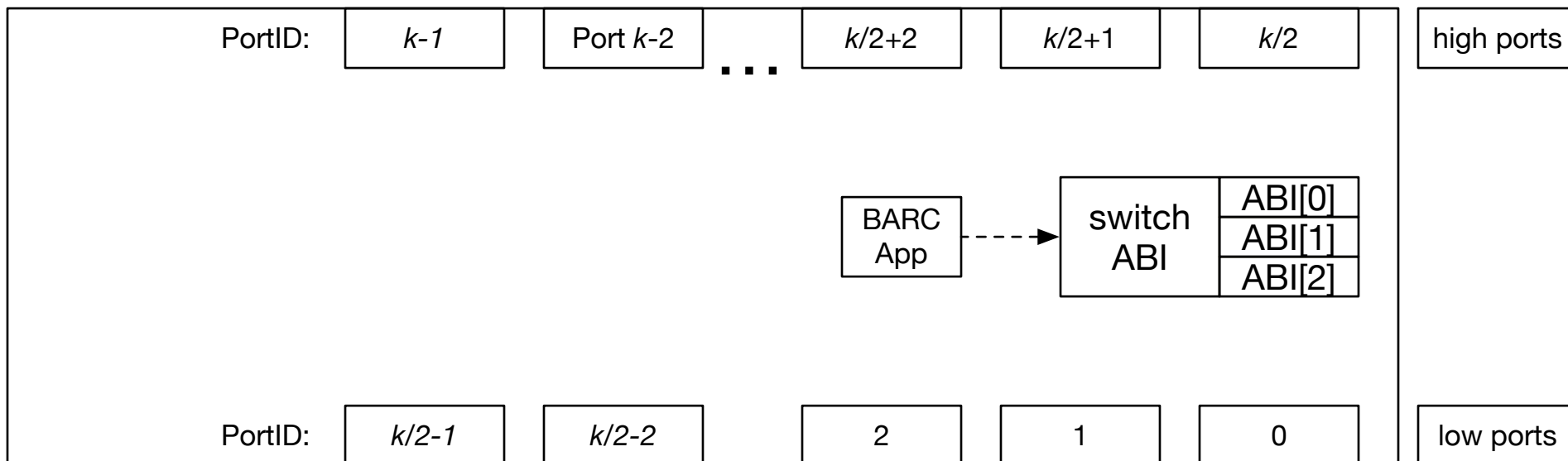
example: $k=6$



three-level k -ary Clos Fat-tree Numerology (example: $k=4$)



Switch Anatomy: Any Switch in Fat-Tree



Each switch is configured identically and runs the same code.

The BARC App is specified within the switch (e.g., P4 switch or softswitch such as BMv2) and runs on startup, initiated by frames originating from the hosts.

BARC results in assigning three switch Address Block Identifier (ABI) bytes to differentiate the switch and customize its behavior. These ABI bytes are retained in switch memory.

Address Blocks

		spine switch (SS)	fabric switch (FS)	rack switch (RS)	host (H)
MSB	AB[0]	0xBE (unicast) 0xBF (multicast)	0xFE (unicast) 0xFF (multicast)	0xEE (unicast) 0xEF (multicast)	0xAE (unicast) 0xAF (multicast)
	AB[1]	Spine Switch ID (sw)	Pod ID (p)	Pod ID (p)	Pod ID (p)
	AB[2]	Spine ID (s)	Spine ID (s)	Rack ID (r)	Rack ID (r)
	AB[3]	*	*	*	Host ID (h)
	AB[4]	*	*	*	* sw (route)
LSB	AB[5]	*	*	*	* s (route)

Note that multicast addresses are conventionally considered to be assigned to an application; devices interested in receiving frames of that application will subscribe to that address.

However, in BARC, multicast addresses are assigned (in blocks) to a device. In the collective multicast, multicast addresses assigned to SSs are used as Collective Addresses (CAs). The first three bytes of a CA are the ABI of that SS, so the first byte of a CA is 0xBF.

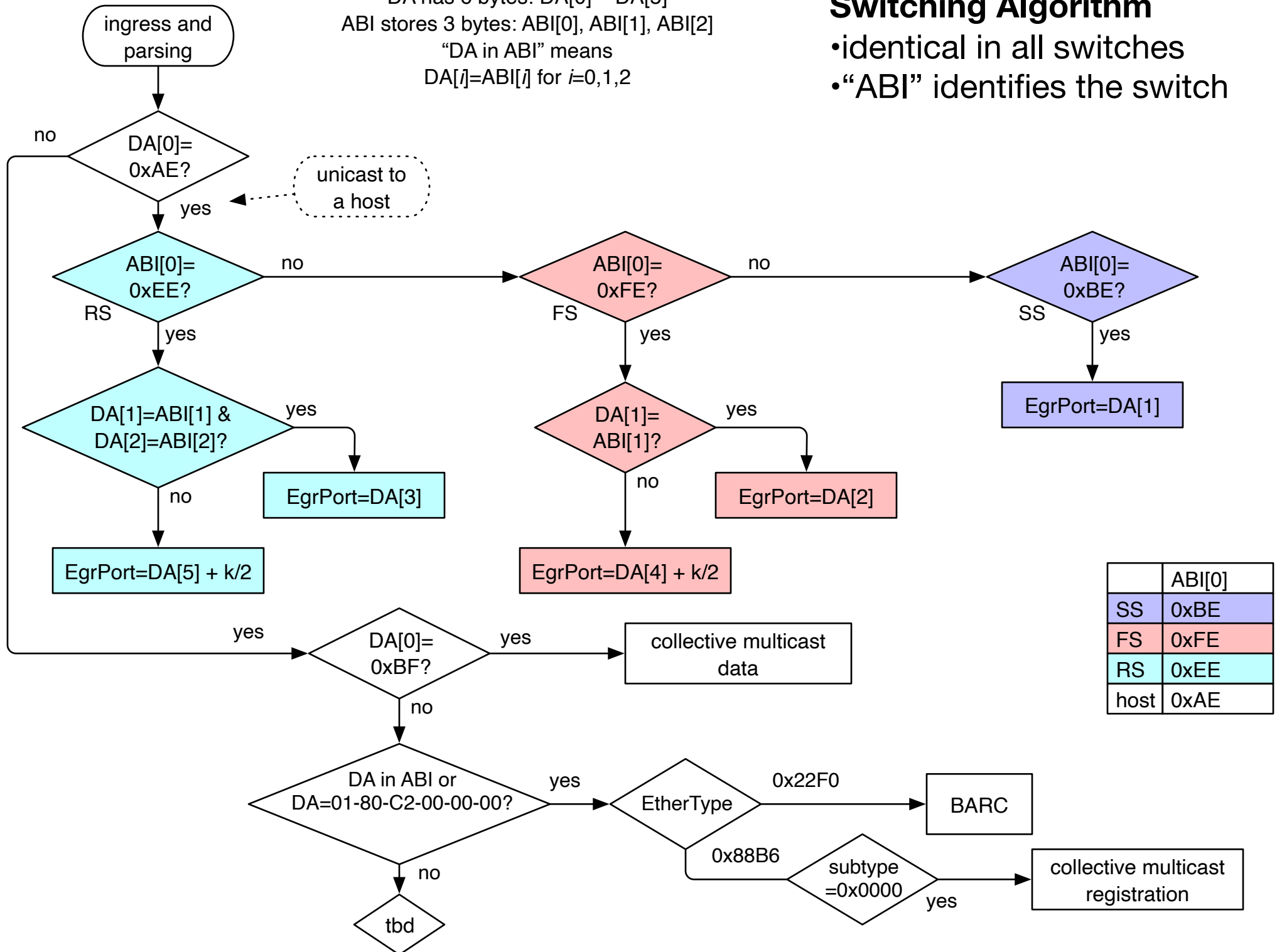
Address Block Identifiers (ABI)

		spine switch (SS)	fabric switch (FS)	rack switch (RS)	host (H)
MSB	ABI[0]	0xBE	0xFE	0xEE	0xAE
	ABI[1]	Spine Switch ID (sw)	Pod ID (p)	Pod ID (p)	Pod ID (p)
	ABI[2]	Spine ID (s)	Spine ID (s)	Rack ID (r)	Rack ID (r)
LSB	ABI[3]				Host ID (h)

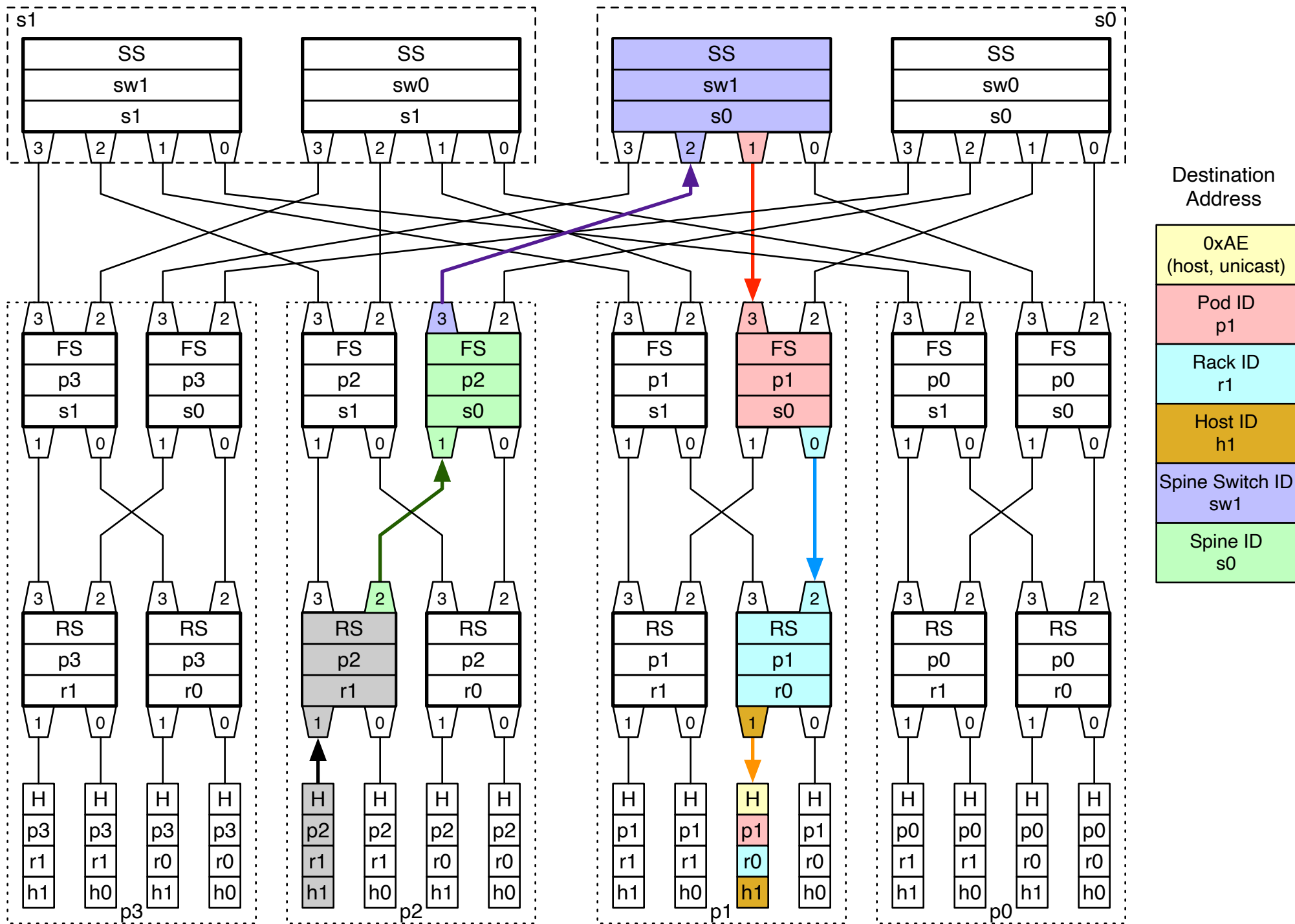
DA has 6 bytes: DA[0] – DA[5]
 ABI stores 3 bytes: ABI[0], ABI[1], ABI[2]
 “DA in ABI” means
 DA[i]=ABI[i] for i=0,1,2

Switching Algorithm

- identical in all switches
- “ABI” identifies the switch



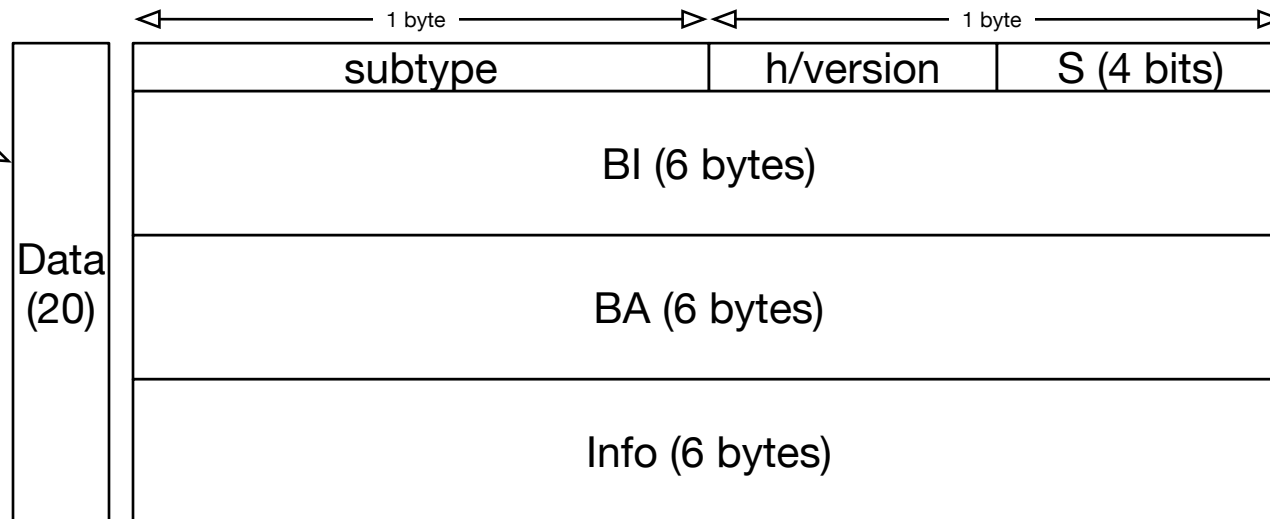
Stateless Unicast forwarding (fully source-specified)



BARC Frame Format

frame field	bytes	content
DA	6	01-80-C2-00-00-00
SA	6	0E-00-00-00-00-00
EtherType	2	0x22F0
Data	20	BARC Data
FCS	4	frame check

the standard
non-forwarding
“nearest customer
bridge” address



Two types of BARC Message: Inquiry and Proposal

BARC Inquiry	DA	01-80-C2-00-00-00
	SA	0E-00-00-00-00-00
	S	I (0xA)
	BI	PABI
	BA	0
	Info	0

BARC Proposal	DA	01-80-C2-00-00-00
	SA	0E-00-00-00-00-00
	S	P (0xB)
	BI	PABI
	BA	0
	Info	0

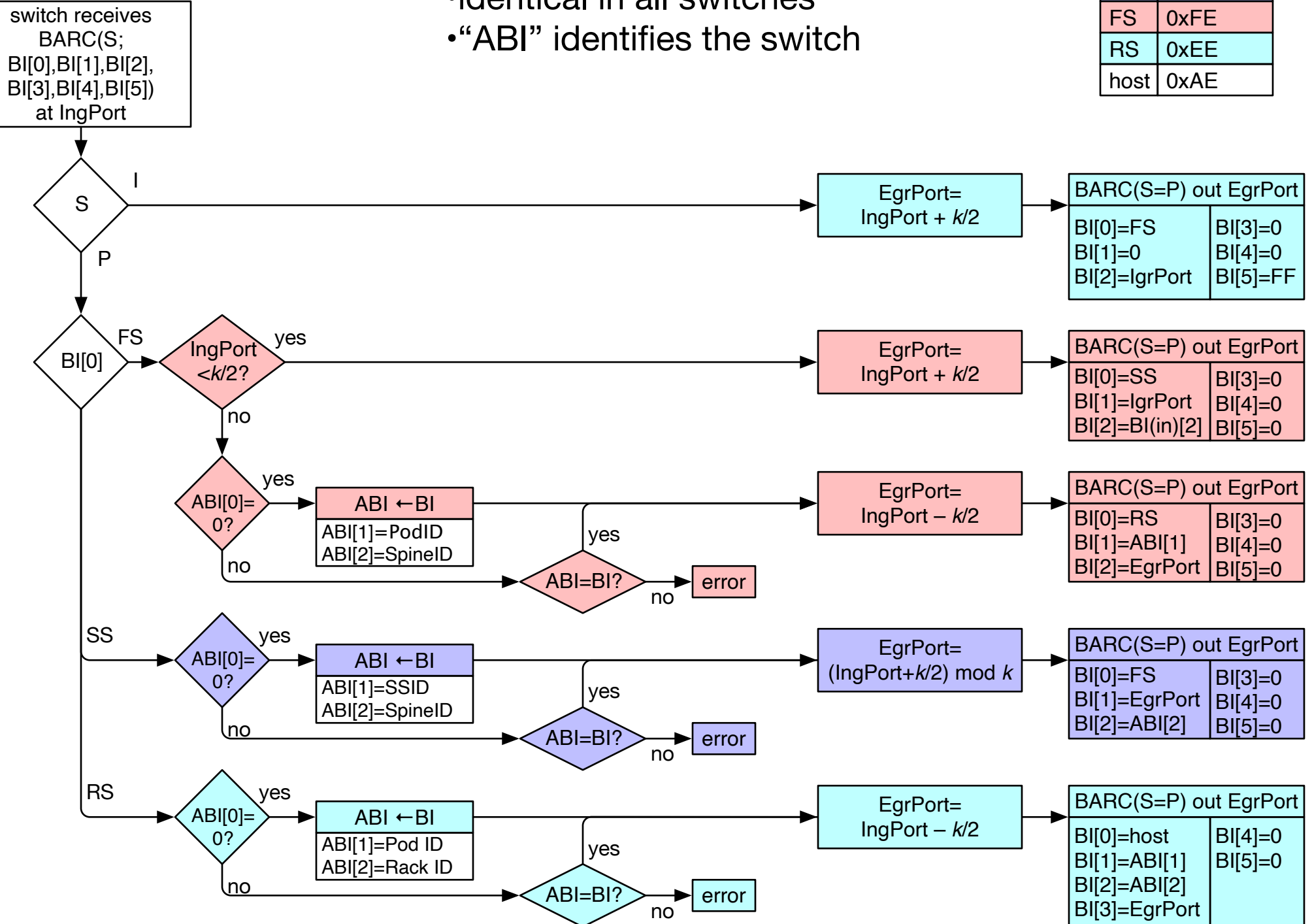
based on
IEEE P802.1CQ/D0.8
Table 15

each host begins by sending a BARC Inquiry with BI[0]=host

BARC Algorithm

- identical in all switches
- “ABI” identifies the switch

	ABI[0]
SS	0xBE
FS	0xFE
RS	0xEE
host	0xAE



Multicast: Forwarding Database and Egress Vector

match ingress frame to a Forwarding Table entry

- exact match of DA, and
- match of any ingress port on list

Egress Vector: binary valued, dimension k

$\text{Egress}[i] = 1$ means frame needs to be forwarded from port i

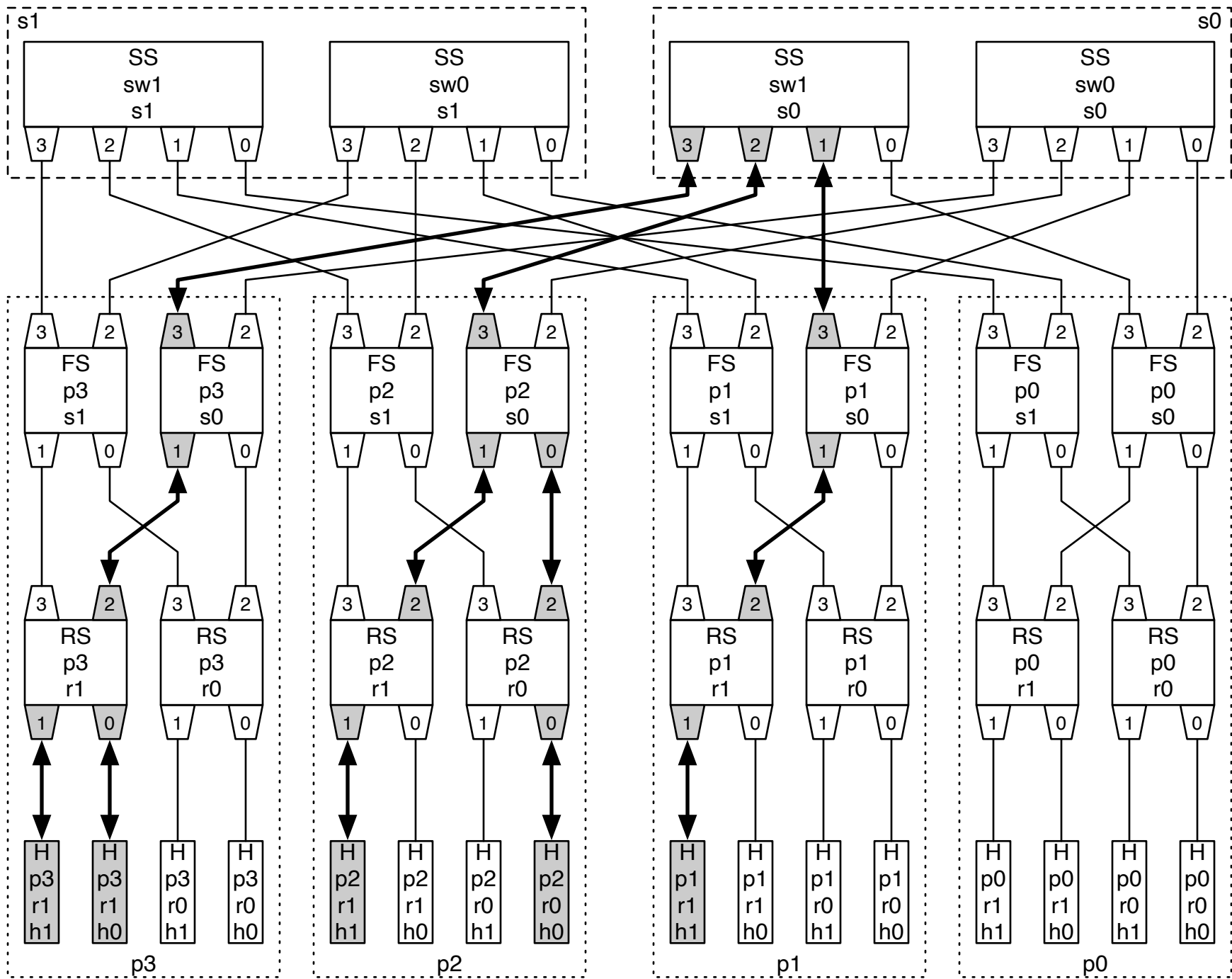
physical ports: 0 through $k-1$

{local (virtual) port: could be $\text{Egress}[k]$, but we don't need to implement that}

consider using P4 Groups

Example Forwarding Table:	multicast frame DA	allowed ingress port list	Egress[0]	Egress[1]	Egress[2]	...	Egress[$k-1$]
	a	1,2	1	1	1	...	1
	b	3,4,5	0	0	0	...	1
	c	0	1	1	1	...	0
	d	2	0	0	0	...	1
	e	4,3	0	0	1	...	0
	f	2	1	1	0	...	0

Collective Multicast



Collective Data Frame

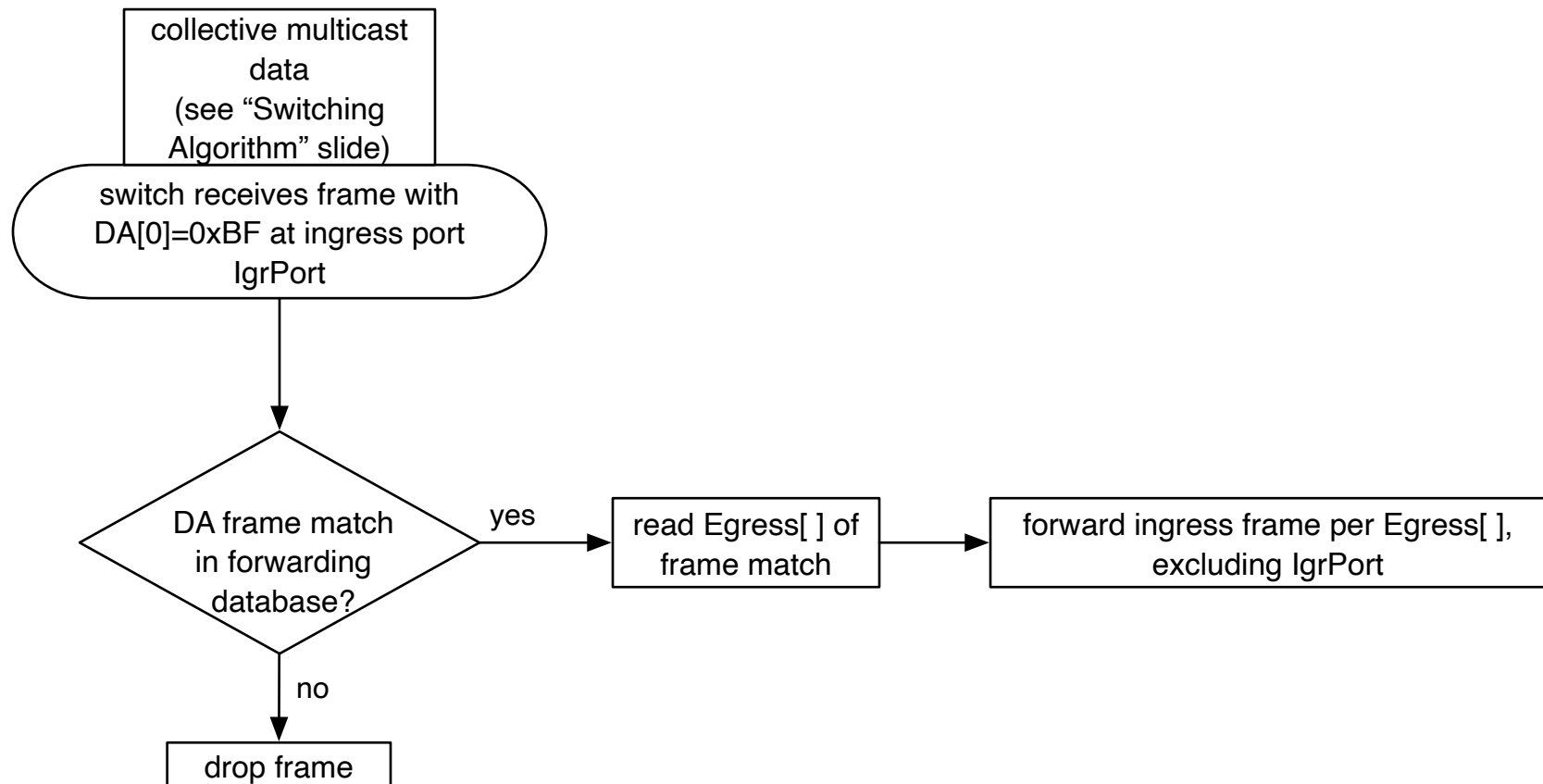
frame field	bytes	content
DA	6	CA
SA	6	host source address
EtherType	2	0x88B6
subtype	2	operation indicator Op
data type	2	DT
data length	2	DL
data value	DL	DV
FCS	4	frame check

	CA	msb	lsb
MSB	CA[0]	0xBF (multicast)	
	CA[1]	Spine Switch ID (<i>sw</i>)	
	CA[2]	Spine ID (<i>s</i>)	
	CA[3]	Ctrl	*
	CA[4]	*	
LSB	CA[5]	*	

Ctrl	
11	multicast
00	operation
10	control
10	(reserved)

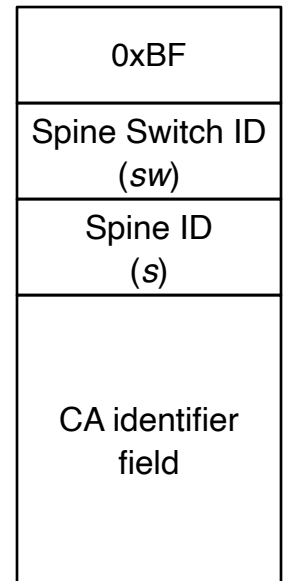
Collective Multicast - Data

- unreliable case!



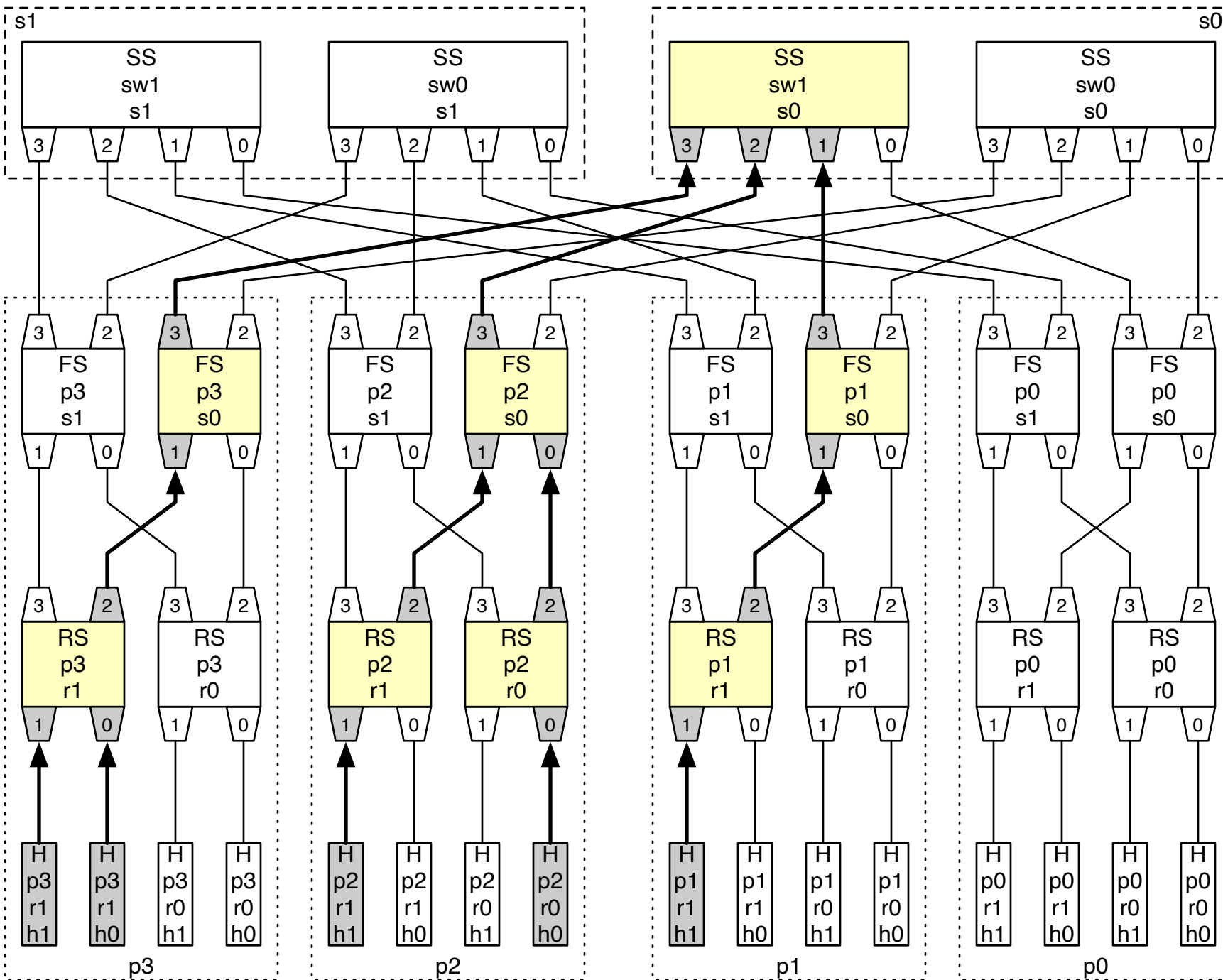
Collective Multicast – Registration

CA address:
6 bytes



↑ SS
ABI
↓

•any frame from a
CA member with
DA=CA is delivered
to each CA member



Collective Registration Frame Format

frame field	bytes	content
DA	6	01-80-C2-00-00-00
SA	6	host source address
EtherType	2	0x88B6
subtype	2	0x0000
CA	6	address
FCS	4	frame check

collective registration: update forwarding database

